

# 自然语言处理 (2025 春季学期)

## 八、机器翻译



# 大纲

- 机器翻译概述
- 基于统计的机器翻译方法
- 基于神经网络的机器翻译方法
- 机器翻译语料库





# 机器翻译概述

根据联合国统计，目前世界上正在使用的语言约有**6000种**，教育系统和公共领域中使用到的语言也有数百种之多。

Warren Weaver就在1947年提出了利用计算机翻译人类语言的可能。

**机器翻译 (Machine Translation)** 是指利用计算机**将一种语言**（源语言）**自动翻译为另外一种语言**（目标语言）的过程。机器翻译是自然语言处理中研究历史最长也最重要的任务之一。

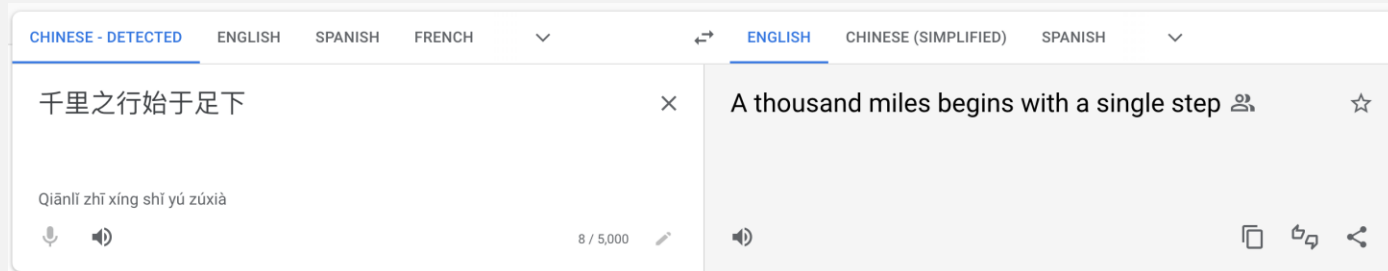


# 机器翻译概述

**机器翻译 (Machine Translation, MT)** 这一概念拥有很长的历史，相关领域的研究最早可以追溯到17世纪。1629年Descartes等人就提出使用统一符号表达不同语言中的同一概念的语义。

现代机器翻译的研究始于上世纪五十年代，Bar-Hillel等人在1951年就开始了机器翻译的研究，并在1952年组织了第一届国际机器翻译会议（International Conference on Machine Translation）。

**机器翻译**的任务定义相对简单，目标就是通过计算机将源语言（Source Language）翻译为目标语言（Target Language）。





# 机器翻译发展历程

机器翻译的发展历程基本代表了自然语言处理领域的发展过程，迄今为止，机器翻译的研究与发展大体上经历了三次主要的浪潮：**基于规则的机器翻译**、**基于统计的机器翻译**以及**基于神经网络的机器翻译**。

**基于规则的机器翻译**是机器翻译任务的第一套解决方案，它基于“**每一种语义在不同的语言当中都存在与其相对应的符号**”这一假设。对于某种语言中的大多数单词而言，通常都能够在另一种语言当中找到表达相同含义的对应的单词。

翻译过程通常被看作一个源语言的词替换过程。之所以被称为“基于规则的方法”，是因为同一种语义在不同的语言当中通常会以不同的词序去表达，词替换过程相对应地需要两种语言的句法规则作为指导。源语言中的每一个单词需要被放置在目标语言中相对应的位置。

基于规则的机器翻译方法的理论非常简洁清晰，但在实践中的性能却不尽如人意。这是由于选择与给定源语言相适配的句法规则在计算上非常低效。同时，为了应对多样的语言现象，语言学家们设计了规模庞大的句法规则。但是，这些规则很难被有效地组织，甚至会出现不同规则相互矛盾的情况。



# 机器翻译发展历程

**基于统计的机器翻译：**在过去的20年以来，统计机器翻译（Statistical Machine Translation, SMT）已经成为机器翻译领域的主流方法，并在工业界得到了广泛的实际应用。

统计机器翻译完全从**数据驱动的角度建模机器翻译任务**。具体来说，通过对双语语料库的**统计找到表达相同含义的单词或短语**。给定一个源语言句子，统计机器翻译首先将其分割成若干个子句，接下来每个部分可以被目标语言的单词或短语替代。

统计机器翻译中最主流的方法是**基于词的统计机器翻译**（Word-based MT）以及**基于短语的统计机器翻译**（Phrase-based SMT），总体包含预处理、句子对齐、词对齐、短语抽取、短语特征准备、语言模型训练等步骤。

**基于神经网络的机器翻译：**神经网络方法在机器翻译任务上的第一次成功应用是Kalchbrenner和Blunsom等人在2013年提出的基于递归神经网络的方法。

神经机器翻译模型在对语言学知识的依赖更少的前提下达到与之前方法相媲美的性能。



# 机器翻译发展历程

机器翻译在经历了几十年的发展后，特别是深度神经网络有效应用于机器翻译，使得模型机器翻译的效果有了很大的提高，**在特定条件下**机器翻译的效果已经能够达到非常好的效果，甚至可以接近人工翻译效果。然而，**在开放环境中，翻译效果还远没有达到直接使用的程度**。根据机器翻译权威评测WMT21给出的人工评测结果，在新闻领域最好的中文到英文翻译系统评分也仅有**75分左右**（满分100分）。机器翻译完全代替人工翻译还有很长的道路。

以王佐良先生对Samuel Ullman所著的《Youth》译文为例：

原文：Youth is not a time of life; it is a state of mind; it is not a matter of rosy cheeks, red lips and supple knees; it is a matter of the will, a quality of the imagination, a vigor of the emotions; it is the freshness of the deep springs of life.

机器翻译结果：青春不是生命的时光；这是一种心态；这不是红润的脸颊、红润的嘴唇和柔软的膝盖；这是意志的问题，是想象力的质量，是情感的活力；它是生命深泉的清新。

王佐良译文：青春不是年华，而是心境；青春不是桃面、丹唇、柔膝，而是深沉的意志，恢宏的想象，炙热的感情；青春是生命的深泉在涌流。





# 机器翻译发展历程

机器翻译虽然经过很多年的发展，目前在特定应用场景下已经能够有很好的效果，整个翻译的效果距离人工翻译“**信达雅**”的要求还是有很大的差距，仍然面临如下挑战：

**(1)自然语言复杂度高：**自然语言具有高度的复杂性、概括性以及多变性，并且是在不断发展的过程中。虽然目前已经有深度神经网络模型参数量达到了1.75万亿，但是相比于自然语言的复杂度来说还是相差很多。

**(2)翻译结果不可解释：**目前机器翻译算法多采用数据驱动的方法，所采用的模型通常不具备可解释性。这就造成了机器翻译算法虽然给出了翻译结果，并且效果可能还很好，但是其对语言的理解和翻译过程与人的理解和翻译过程完全不同。

**(3)翻译结果评测困难：**语言有很大的灵活性和多样性，同样一句话可以有非常多种的翻译方法。对机器翻译性能进行评测可以采用人工评测和半自动评测方法。





# 大纲

---

- 机器翻译概述
- 基于统计的机器翻译方法
- 基于神经网络的机器翻译方法
- 机器翻译语料库





# 基于统计的机器翻译方法

---

机器翻译任务到今天已经经历了长达数十年的发展历史。

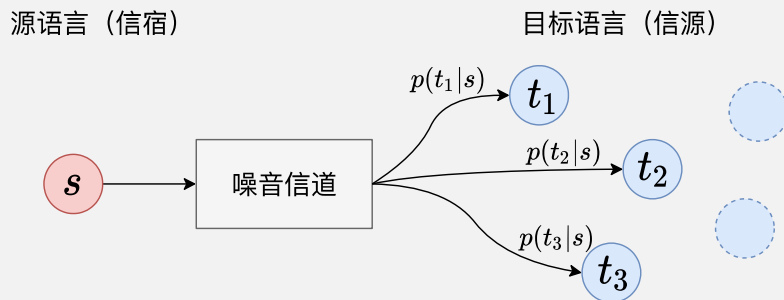
**在很长一段时间之内，基于统计的机器翻译**方法在学术界受到了许多关注并在工业界得到了广泛地应用。

尽管这些模型在当下已经不再是人们关注的焦点，回顾这些方法仍然对我们完整地了解机器翻译的过去以及展望未来的研究具有十分重要的意义。



# 任务定义与基本问题

IBM机器翻译模型构建在噪声信道模型（Noise Channel Model）的基础之上，其模型基本框架如下图所示。



源语言句子 $s$ 和候选的目标语言句子 $t_i$ 通过一个噪声信道相连，在已知 $s$ 和噪声信道性质的前提之下，就能够得到信源，**也即目标语言的分布 $P(t|s)$** 。机器翻译的过程本质上就是在给定源语言句子 $s$ 的前提下，从分布 $P(t|s)$ 找到最有可能的目标语言 $t$ 作为输出，这一搜索过程也被称为解码。

$$\hat{t} = \operatorname{argmax}_t P(t|s)$$



# 任务定义与基本问题

在上述噪声信道基础框架下，统计机器翻译任务需要解决如下**三个核心的基本问题**：

**问题1：建模** 如何通过易于被计算机处理的数学模型对 $P(t|s)$ 进行合理地建模以刻画源语言和目标语言之间的关系。

**问题2：训练** 如何从给定的平行语料库（即源语言-目标语言对组成的语料集合）中获得最优的模型参数。

**问题3：解码** 如何从模型 $P(t|s)$ 中搜索出最优的目标语言序列 $t$ 。



# 任务定义与基本问题

统计机器翻译模型的核心在于**对 $P(t|s)$ 的定义**，这一定义决定了模型性能的上限并且也是后续训练和解码的基础。IBM模型通过贝叶斯公式对这一翻译概率做如下变换：

$$P(t|s) = \frac{P(s, t)}{P(s)} = \frac{P(s|t)P(t)}{P(s)}$$

通过上述变换，**翻译模型  $P(t|s)$  被分解为了三个部分**：

- (1) 从目标语言指向源语言的翻译概率 $P(s|t)$ ；
- (2) 目标语言的语言模型 $P(t)$ ；
- (3) 源语言序列语言模型 $P(s)$ 。

需要注意的是，通过贝叶斯变换， $P(t|s)$  和  $P(s|t)$  只是翻译的方向不同，建模难度并没有下降。其核心是为了引入目标语言的语言模型。这是由于IBM模型本质上是一种基于词的统计机器翻译模型，仅通过翻译概率 $P(s|t)$  很难有效地建模目标语言单词之间的相对位置关系，也即目标语言序列的流畅程度。





# 任务定义与基本问题

$P(s)$  是一个不变量，不会影响到最终的解码过程，在建模的过程当中 $P(s)$ 通常不需要被计算，可以省略：

$$\hat{t} = \operatorname{argmax}_t P(t|s) = \operatorname{argmax}_t \frac{P(s|t)P(t)}{P(s)} = \operatorname{argmax}_t P(s|t)P(t)$$

基于上述分析，IBM模型的建模问题转换为如何建模翻译概率 $P(s|t)$ 以及语言模型 $P(t)$ 。翻译概率 $P(s|t)$ 主要用于衡量源语言和目标语言之间的匹配程度。然而，自然语言拥有极其庞大的潜在的组合方式。假设某种语言对应的词表大小为10000，那么一个简单的长度为10的句子就对应着 $10000^{10} = 10^{40}$ 种不同的组合方式。

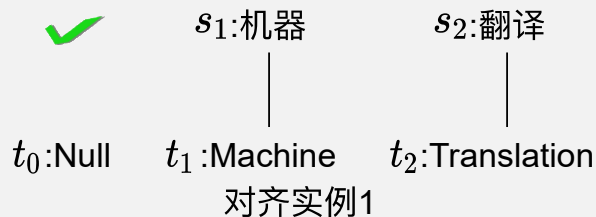
基于任何已有的平行语料库，直接在句子层级对上述翻译概率进行估计，都会面临严重的数据稀缺问题。因此，IBM模型将句子层级的翻译概率进一步拆解为单词级别的对应关系的组合，从而缓解上述数据稀疏的问题，这一拆解过程又被称为词对齐。



# 任务定义与基本问题

词对齐作为IBM模型构建的重要基础之一，描述了目标语言和源语言之间单词级别的对应关系。

以下图中的对齐实例为例，给定源语言文本“机器 翻译”，对应的目标语言翻译为“Machine Translation”。其中，“机器”一词对应“Machine”而“翻译”一词对应“Translation”。



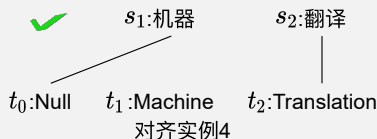
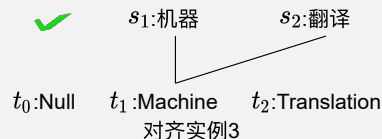
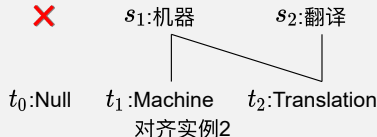
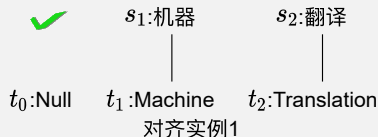
使用记号 $\mathbf{a}=\{a_1, \dots, a_m\}$ 表示这种对应关系，其中 $a_j$ 表示源语言中的单词 $s_j$ 和目标语言中的单词 $t_{a_j}$ 存在对应关系。举例来说，在上述对齐实例中， $a_1=1$   $a_2=2$ 。



# 任务定义与基本问题

为了建模方便，IBM模型对**词对齐**做了如下两个限制：

- 对于每一个源语言单词，至多只能对齐到一个目标语言单词上。下图对齐实例2中，源语言单词“机器”同时对应到了两个目标语言单词“Machine”和“Translation”，这就违反了上述IBM模型假设。而其余的对齐实例均满足这一假设。
- 存在一些源语言单词，它们可以对齐到一个额外增设的虚拟目标语言单词“Null”上，也即对空。下图的对齐实例4中的“机器”一词就对应到了目标语言的“Null”上。对空情况的额外考虑并不是没有意义的。事实上，对空的现象在翻译的过程当中频繁出现，如虚词的翻译。







# 任务定义与基本问题

IBM模型认为句子级别的翻译概率可以**通过单词级别的翻译概率组合而成**，并将**词对齐作为一种隐变量**整合到翻译概率的建模过程中。这样，原本较为困难的句子级别的建模问题就被分解为一个分步学习的问题：

$$P(s|t) = \sum_a P(s, a|t)$$

其中， $s = \{s_1, s_2, \dots, s_m\}$ 表示一个长度为 $m$ 的源语言序列， $t = \{t_1, t_2, \dots, t_l\}$ 表示一个长度为 $l$ 的目标语言序列， $a = \{a_1, a_2, \dots, a_m\}$ 表示源语言中每一个单词 $s_j$ 对应的目标语言单词序号 $a_j$ 。

直接建模 $P(s|t)$ 仍然非常复杂，为了解决这个问题，IBM模型对上述概率通过链式法则做了进一步展开，并为后续的简化做了准备。

$$P(s, a|t) = P(m|t) \prod_{j=1}^m P(a_j | a_1^{j-1}, s_1^{j-1}, m, t) P(s_j | a_1^j, s_1^{j-1}, m, t)$$

其中， $a_1^{j-1}$  表示源语言序列中前 $j-1$ 个单词的词对齐， $s_1^{j-1}$  表示源语言序列中的前 $j-1$ 个单词。



# 任务定义与基本问题

$$P(s, a|t) = P(m|t) \prod_{j=1}^m P(a_j|a_1^{j-1}, s_1^{j-1}, m, t) P(s_j|a_1^j, s_1^{j-1}, m, t)$$

这一展开看似较为复杂，实际上**每个部分都具有较为清晰的物理含义**。

给定一个目标语言序列  $t$ ，首先通过概率  $P(m|t)$  估计源语言序列的长度  $m$ 。

接下来，通过  $m$  次循环从**左向右依次生成源语言序列和它们的词对齐**。

在第  $j$  次循环当中，首先通过目标语言序列  $t$ ，前  $j-1$  次循环中生成的词对齐序列以及源语言序列  $s_1^{j-1}$  产生当前位置的词对齐  $a_j$ ，即

接下来结合  $a_j$  进一步生成当前位置的源语言单词  $s_j$ ，也即

至此，翻译概率的建模实际上就被转换为源语言文本和词对齐的生成问题





# 任务定义与基本问题

翻译概率的建模实际上就被转换为源语言文本和词对齐的生成问题。但是，**仍然还存在两个迫切需要解决的问题：**

(1) 为了最终实现对翻译概率  $P(s|t)$  的建模，在公式中需要对所有可能的词对齐进行求和。然而，可能的词对齐的数量随着源语言序列的长度呈指数级别增长，如何计算这一求和式是第一个需要被解决的问题。

(2) 公式中通过链式分解为建模  $P(s,a|t)$  提供了一种可行的方向，然而如何通过目标语言序列估计源语言序列的长度  $P(m|t)$ ，以及如何建模源语言  $P(s_j|a_1^j, s_1^{j-1}, m, t)$  和词对齐的生成过程  $P(a_j|a_1^{j-1}, s_1^{j-1}, m, t)$  尚待解决。

对于上述两个问题的解决实际上对应着**五个不同的IBM模型**



# 任务定义与基本问题

在完成了对翻译概率  $P(s|t)$  以及语言模型  $P(t)$  的建模与优化之后，下一个需要解决的问题就是解码，

## 代码 8.1: 贪婪解码算法

```
输入: 源语言句子序列  $s = \{s_1, s_2, \dots, s_m\}$ 
输出: 解码出的目标语言序列  $t = \{t_1, t_2, \dots, t_l\}$ 
初始化一个空集  $t_{best} = \emptyset$  记录解码出的最优目标语言序列;
初始化一个布尔数组  $u$ , 记录每个源语言单词是否已经被解码;
对于每个源语言单词  $s_i$ , 获取其对应单词级别目标语言候选  $\pi$ ;
for  $i = 1$  to  $m$  do
    初始化一个空集  $h = \emptyset$  记录当前最优解码序列的扩展;
    for  $j = 1$  to  $m$  do
        if  $u[j]$  为假, 也即当前单词没有被解码过 then
            将当前单词的候选翻译  $\pi[j]$  添加到  $h$  当中;
        end
    end
    从当前的最优翻译扩展候选  $h$  中找到最优的翻译结果, 并记为  $t_{best}$ ;
    这一最优结果所对应的源语言单词被记录为已解码  $u[j] = True$ ;
end
return  $t_{best}$ 
```





# IBM 模型 I

$$P(\mathbf{s}, \mathbf{a} | \mathbf{t}) = P(m | \mathbf{t}) \prod_{j=1}^m P(a_j | \mathbf{a}_1^{j-1}, \mathbf{s}_1^{j-1}, m, \mathbf{t}) P(s_j | \mathbf{a}_1^j, \mathbf{s}_1^{j-1}, m, \mathbf{t})$$

IBM模型 I 假定源语言句子序列长度的生成概率服从均匀分布，即：

$$P(m | \mathbf{t}) \equiv \epsilon$$

源语言中的每一个单词被认为是等可能地和目标语言中的所有单词对齐

$$P(a_j | \mathbf{a}_1^{j-1}, \mathbf{s}_1^{j-1}, m, \mathbf{t}) \equiv \frac{1}{l+1}$$

当对齐关系明确之后，IBM模型I 假设当前时刻源语言单词 $s_j$ 的生成只依赖于和它对齐的目标语言单词 $t_{a_j}$ ：

$$P(s_j | \mathbf{a}_1^j, \mathbf{s}_1^{j-1}, m, \mathbf{t}) \equiv f(s_j | t_{a_j})$$



# IBM 模型 I

经过上述三个部分的化简，翻译概率  $P(s|t)$  可以按照下面的方式得到：

$$\begin{aligned} P(s|t) &= \sum_a P(s, a|t) \\ &= P(m|t) \prod_{j=1}^m P(a_j | \mathbf{a}_1^{j-1}, \mathbf{s}_1^{j-1}, m, t) P(s_j | \mathbf{a}_1^j, \mathbf{s}_1^{j-1}, m, t) \\ &= \sum_a \frac{\epsilon}{(l+1)^m} \prod_{j=1}^m f(s_j | t_{a_j}) \end{aligned}$$

观察 IBM 模型 I 最终的建模结果可以发现，翻译概率  $P(s|t)$  最终变成了在所有可能的词对齐的基础上，对单词对翻译概率的连乘。



# IBM 模型 I

另一个需要解决的问题是对齐序列  $a$  的求和问题。一个长度为  $m$  的源语言序列的每一个单词有可能对齐到长度为  $l+1$  的目标语言的任何一个位置上。

$$P(s|t) = \sum_a P(s, a|t) = \underbrace{\sum_a \frac{\epsilon}{(l+1)^m}}_{(l+1)^m \text{次循环}} \underbrace{\prod_{j=1}^m f(s_j|t_{a_j})}_{m \text{次循环}}$$

在实际的计算过程中，IBM 模型采用如下的计算技巧：

$$\begin{aligned} P(s|t) &= \sum_a \frac{\epsilon}{(l+1)^m} \prod_{j=1}^m f(s_j|t_{a_j}) \\ &= \frac{\epsilon}{(l+1)^m} \sum_a \prod_{j=1}^m f(s_j|t_{a_j}) \\ &= \frac{\epsilon}{(l+1)^m} \prod_{j=1}^m \sum_{i=1}^l f(s_j|t_i) \end{aligned}$$





# IBM 模型 I

此处的计算技巧通过将若干个连乘结果的加和转换为若干加和结果的连乘。计算复杂度由原本的 $O((l+1)^m \cdot m)$ 降低为 $O((l+1) \cdot m)$

$$\begin{aligned} & f(1,0)f(2,0) + f(1,0)f(2,1) + f(1,0)f(2,2) + \\ & f(1,1)f(2,0) + f(1,1)f(2,1) + f(1,1)f(2,2) + \\ & f(1,2)f(2,0) + f(1,2)f(2,1) + f(1,2)f(2,2) \\ &= \sum_{a_1=0}^2 \sum_{a_2=0}^2 f(1, a_1) f(2, a_2) \\ &= \sum_{a_1=0}^2 \sum_{a_2=0}^2 \prod_{j=1}^2 f(j, a_j) \\ &= (f(1,0) + f(1,1) + f(1,2)) (f(2,0) + f(2,1) + f(2,2)) \\ &= \prod_{j=1}^2 \sum_{i=0}^2 f(j, i) \\ &\sum_{a_1=0}^l \cdots \sum_{a_m=0}^l \prod_{j=1}^m f(s_j | t_{a_j}) = \prod_{j=1}^m \sum_{i=0}^l f(s_j | t_i) \end{aligned}$$







# IBM 模型 I

IBM模型I 的优化过程本质上基于极大似然估计的思想，也即找到一组参数  $\hat{\theta} = \{f(s_x|t_y)\}$ ，使得模型  $P_{\theta}(s|t)$  能够对训练集中的句对  $(s,t)$  输出尽可能大的概率，形式化的描述如下：

$$\begin{aligned}\hat{\theta} &= \operatorname{argmax}_{\theta} P_{\theta}(s|t) \\ &= \operatorname{argmax}_{\theta} \frac{\epsilon}{(l+1)^m} \prod_{j=1}^m \sum_{i=1}^l f(s_j|t_i) \\ \text{s.t. } &\forall t_y, \sum_{s_x} f(s_x|t_y) = 1\end{aligned}$$

利用拉格朗日乘子法将上式转换为无约束优化问题，得到如下无约束的优化目标：

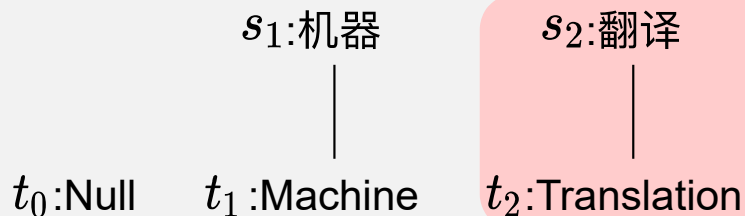
$$L(f, \lambda) = \frac{\epsilon}{(l+1)^m} \prod_{j=1}^m \sum_{i=1}^l f(s_j|t_i) - \sum_{t_y} \lambda_{t_y} \left( \sum_{s_x} f(s_x|t_y) - 1 \right)$$





## IBM 模型 II

IBM模型I 虽然很好地简化了模型的复杂程度使得翻译的建模成为了可能，但其中的一些简化与真实情况存在着较大的差异，导致翻译性能受到了较大的限制。**最突出的问题是词对齐的概率服从均匀分布。**



IBM模型I 对这一问题作出了修正，它认为词对齐存在着一定的倾向性。具体来说，IBM模型I 假设源语言单词  $x_j$  的对齐位置  $a_j$  的生成概率与它所在的位置  $j$  和源语言序列长度  $m$  以及目标语言序列长度  $l$  有关，形式化表示为：

$$P(a_j | \mathbf{a}_1^{j-1}, \mathbf{s}_1^{j-1}, m, t) \equiv a(a_j | j, m, l)$$



## IBM 模型 II

$a(a_j|j, m, l)$  表示源语言序列中第  $j$  个位置词对齐的生成概率，它被建模为IBM模型II中的一个需要学习的参数。除了词对齐假设之外，其余的模型假设均与IBM模型I 相同，将新的词对齐生成概率按照上一小节所述的建模过程能够得到IBM模型II 的翻译建模表达式为：

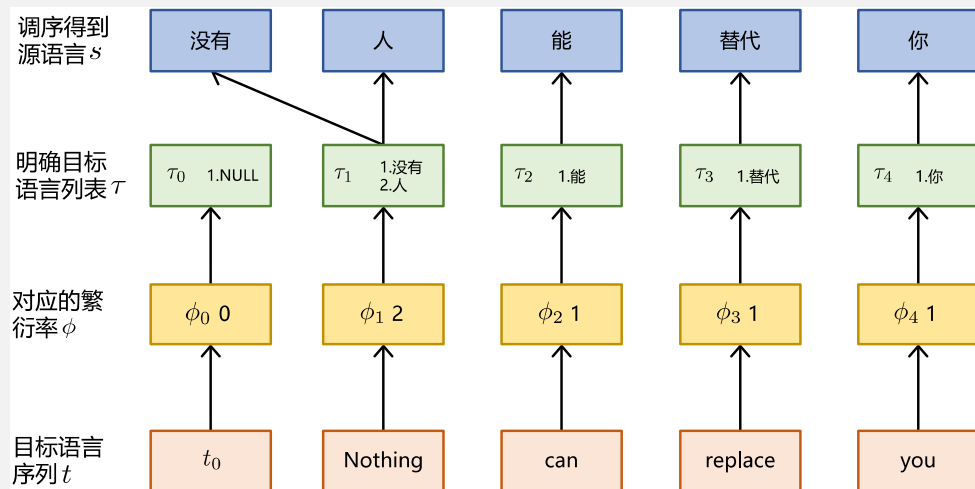
$$P(\mathbf{s}|\mathbf{t}) = \epsilon \prod_{j=1}^m \sum_{i=1}^l a(i|j, m, l) f(s_j|t_i)$$



# IBM 模型 III

IBM模型I和II 存在的一个共同的问题是将单词翻译的过程建模为了一个独立的过程，这就导致它们不能很好地描述**多个源语言单词对齐到同一个目标语言单词**的情况。

**IBM模型III是一种基于繁衍率的模型**，可以在一定程度上解决上述问题。这里的**繁衍率 (Fertility)** 是指每个目标语言单词生成源语言单词的个数。



模型首先确定每个目标语言单词的繁衍率  $\phi_i$

依据繁衍率确定目标语言对应的源语言单词是什么，这样就得到了每个目标语言单词所对应的源语言单词列表



# IBM 模型 III

IBM模型III的翻译概率的形式如下所示:

$$P(\mathbf{s}|\mathbf{t}) = \sum_{\mathbf{a}} \left[ \binom{m-\phi_0}{\phi_0} p_0^{m-2\phi_0} p_1^{\phi_0} \prod_{i=1}^l \phi_i! n(\phi_i | t_i) \cdot \prod_{j=1}^m t(s_j | t_{a_j}) \cdot \prod_{j=1, a_j \neq 0}^m d(j | a_j, m, l) \right],$$
$$s.t. \quad \sum_{s_x} t(s_x | t_y) = 1,$$
$$\sum_j d(j | i, m, l) = 1,$$
$$\sum_{\phi} n(\phi | t_y) = 1,$$
$$p_0 + p_1 = 1$$

详细过程可以参阅文献 [411]

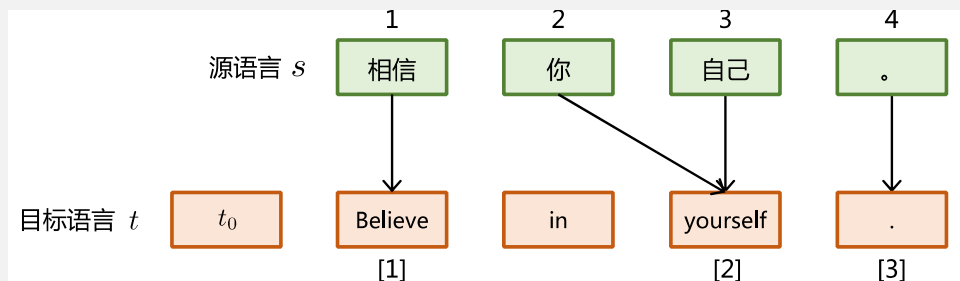


## IBM 模型 IV

当一个目标语言单词对应多个源语言单词时，这些**源语言单词往往会构成一个整体，也即一个短语**。然而前面所述的三个IBM模型并没有对与这种情况做特殊的设计，这就导致了源语言中的单词短语可能会被打散。

针对这个问题，IBM模型IV做出了进一步的修正。它将原本单词之间的对应关系拓宽到了**概念(Concept)**之间的对应。这里的概念是指具有独立语法或语义的一组单词。

IBM模型IV将目标语言的概念约束为那些非空对齐的目标语言单词，且要求所有的目标语言概念都只能由一个单词构成。





## IBM 模型 IV

IBM模型IV所做的修正主要体现在扭曲度的建模，对于  $[i]$  对应的源语言单词列表中的第一个单词  $\tau_{[i]1}$ ，它的扭曲度计算公式如下：

$$P(\pi_{[i]1} = j | \pi_1^{[i]-1}, \tau_0^l, \phi_0^l, t) = d_1(j - \odot_{i-1} | A(t_{[i-1]}), B(s_j))$$

此处的  $\pi_{ik}$  表示目标语言序列中第  $i$  个单词所对应的源语言列表中的第  $k$  个单词的位置。对于列表中其他单词的扭曲度，则使用如下公式进行计算：

$$P(\pi_{[i]k} = j | \pi_{[i]1}^{k-1}, \pi_1^{[i]-1}, \tau_0^l, \phi_0^l, t) = d_{>1}(j - \pi_{[i]k-1} | B(s_j))$$

其中， $A(\cdot)$ 和 $B(\cdot)$ 分别表示从源语言、目标语言单词向单词词类映射的函数。这一扭曲度函数的改进**背后的思想是**，在生成  $t_{[i]}$  的第一个源语言单词时，要考虑平均位置  $\odot_{[i]}$  和这个源语言单词之间的绝对距离。随后生成的单词所放置的位置则要考虑前一个放置完的单词的相对位置，以及当前源语言单词的词类。这个过程实际上，使得同一个目标语言单词所生成的源语言单词之间可以相互影响，从而避免了独立生成各个源语言单词所带来的冲突问题。

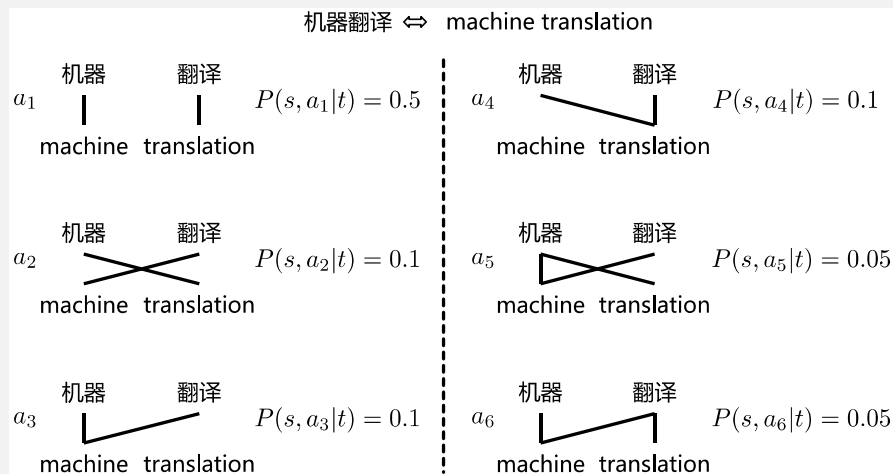




# IBM 模型 V

IBM模型V针对词对齐的过程做了进一步的约束。它认为**同一个源语言单词**不应当由多个目标语言单词转换而来。

如下图所示，前面4种词对齐方式都是合法的。然而，对于词对齐 $a_5$ 和 $a_6$ 来说，源语言单词“机器”和“翻译”分别对应着两个目标语言单词。







# IBM 模型 V

为了约束这种情况的出现，IBM模型V在放置每一个源语言单词时都会检查这个位置是否已经放置了其他单词。

为了实现这一点，引入一个新的变量  $v(j, \tau_1^{[i]-1}, \tau_{[i]1}^{k-1})$ ，它表示在放置  $\tau_{[i]k}$  之前，源语言的前 j 个位置还有多少空余。为了简便起见，后续记这个变量为  $v_j$ 。这样，对于单词 [i] 所对应的源语言单词列表中的第一个单词  $\tau_{[i]1}$  有：

$$P(\tau_{[i]1} = j | \pi_1^{[i]-1}, \tau_0^l, \phi_0^l, t) = d_1(v_j | B(s_j), v_{\odot_{i-1}}, v_m - (\phi_{[i]} - 1)) \cdot (1 - \delta(v_j, v_{j-1})),$$

对于其他单词  $\tau_{[i]k}, 1 < k \leq \phi_{[i]}$ ，有：

$$\begin{aligned} P(\pi_{[i]k} = j | \pi_{[i]1}^{k-1}, \pi_1^{[i]-1}, \tau_0^l, \phi_0^l, t) \\ = d_{>1}(v_j - v_{\pi_{[i]k-1}} | B(s_j), v_m - v_{\pi_{[i]k-1}} - \phi_{[i]} + k) \cdot (1 - \delta(v_j, v_{j-1})) \end{aligned}$$

此处的  $1 - \delta(v_j, v_{j-1})$  是用来判断第 j 个位置是否为空。如果第 j 个位置为空，则  $v_j = v_{j-1}$ ，这样  $P(\pi_{[i]1} = j | \pi_1^{[i]-1}, \tau_0^l, \phi_0^l, t) = 0$ 。这样就避免了词对齐的冲突问题。



# 大纲

---

- 机器翻译概述
- 基于统计的机器翻译方法
- 基于神经网络的机器翻译方法
- 机器翻译语料库





# 基于神经网络的机器翻译方法

**传统机器翻译方法高度依赖于繁杂的特征工程**，合理特征的设计对系统构建者的语言学背景具有较高的要求，同时需要在不断地试错过程中修正。这些特征往往不能够完整地反映输入文本的语义。

- 语言模型作为传统机器翻译模型的重要组成部分，为了降低模型复杂度而引入的马尔可夫假设使得上下文窗口之外的语义依赖无法被建模
- 从输入文本表示的角度来说，经典的词袋模型（bag-of-words, BOW）则忽略了词序对输入文本表示的影响

传统机器翻译方法存在一些**固有缺陷**影响其进一步提升：

- 以最具代表性的基于短语的统计机器翻译方法为例，翻译通过将输入的源语言切分成短语并替换为目标语言的过程完成，**短语范围之外的长程依赖在这一过程中被完全忽略**，进而造成翻译结果中的**错误和不一致性**。
- 同时，为了提升翻译的准确性和流畅度，**越来越多的功能模块不断被设计并添加到统计翻译模型当中**（如语言模型、调序模型、长度调整模型等）。复杂的翻译组件使得系统的整体调优和稳定性受到一定程度的影响。

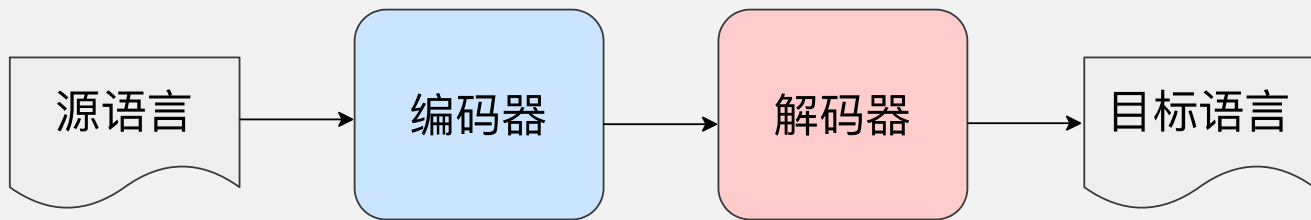


# 基于神经网络的机器翻译方法

神经网络模型作为一个强力的特征抽取器，能够自动地学习输入文本的最优表征，从而在很大程度上减少对领域知识的要求及繁琐的特征工程预处理步骤。

而以循环神经网络、Transformer为代表的神经机器翻译方法能够有效地建模长程依赖，端到端的特性也使得系统的整体结构变得更加紧凑易于调整。

现代神经机器翻译模型大多依据序列到序列的方式对任务进行建模。**给定源语言输入文本**，训练目标是找到**最合适的目标语言**句子作为译文。





# 基于神经网络的机器翻译方法

神经网络模型作为一个强力的特征抽取器，能够自动地学习输入文本的最优表征，从而在很大程度上减少对领域知识的要求及繁琐的特征工程预处理步骤。

而以循环神经网络、Transformer为代表的神经机器翻译方法能够有效地建模长程依赖，端到端的特性也使得系统的整体结构变得更加紧凑易于调整。

现代神经机器翻译模型大多依据序列到序列的方式对任务进行建模。**给定源语言输入文本**，训练目标是找到**最合适的目标语言**句子作为译文。

形式化地来说，给定源语言序列 $S=\{s_1, s_2, \dots, s_n\}$ ，神经机器翻译模型试图找到具有最大条件概率 $P(T|S)$ 的目标语言序列 $T=\{t_1, t_2, \dots, t_m\}$ ， $n$ 和 $m$ 分别表示源语言和目标语言的长度。在生成目标语言句子的每个单词时，源语言和已经生成的目标语言信息会被使用。因此，神经机器翻译的整体过程可以按照如下公式描述：

$$\operatorname{argmax} \prod_{i=1}^m P(t_i | t_{j < i}, S),$$

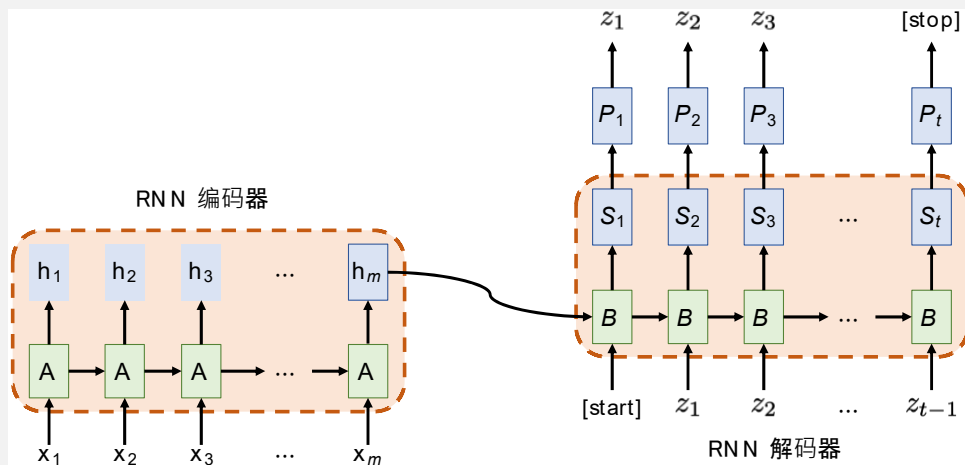


# 循环神经网络翻译模型

神经机器翻译模型大多**基于序列到序列的架构**，完成从源语言到目标语言的转换过程。

不同神经机器翻译模型的**主要区别在于编码器和解码器**所采用的结构上的差异。

自然语言文本可以看做一种时间序列数据，因此一种常见做法是采用基于循环神经网络的结构完成对源语言文本的编码以及目标语言文本的生成。

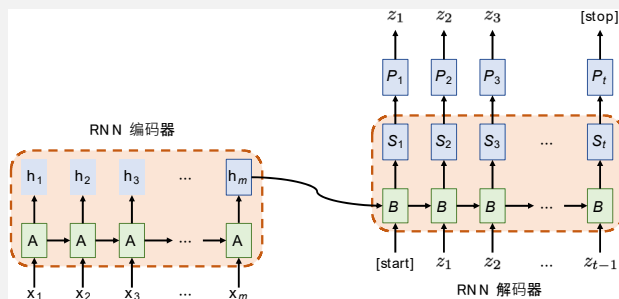




# 循环神经网络翻译模型

**左侧为编码器部分**，源语言单词按照其在文本序列中的先后顺序被依次送入到循环神经网络（RNN）当中。在每个时间步  $t$  中，模型依据送入的源语言单词  $x_t$  对应修改维护其模型内部的隐状态  $h_t$ ，这个隐状态编码了输入的源语言序列前  $t$  个时刻的所有必要信息。按照这种方式当  $m$  个输入全部被送入到编码器之后，所对应的  $h_m$  可以认为**包含了源语言序列的所有信息**。

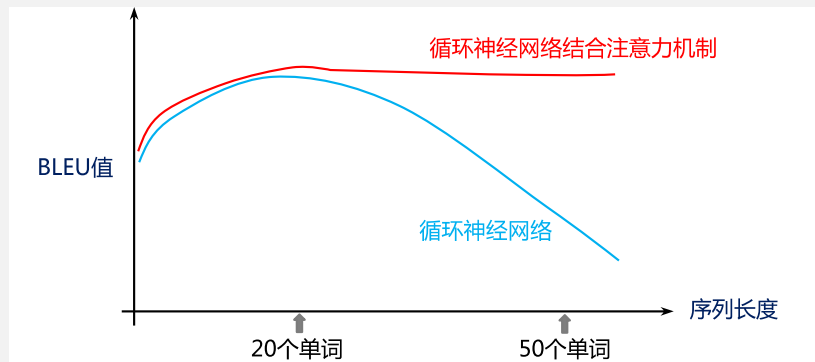
**右半部分是RNN解码器部分**，它接收编码器输出的编码源语言句子信息的向量  $h_m$  作为初始隐状态  $s_0$ 。由于RNN的循环过程在每个时间步都要求一个输入单词，为了启动解码过程，一般会使用一个保留的**特殊符号`[Start]`**作为翻译开始的标记送入到RNN解码器当中，并解码出目标语言序列的**第一个单词  $z_1$** 。接下来， $z_1$  会作为下一个时刻的输入被送入到循环神经网络当中并按照不断迭代产生后续的预测。





# 循环神经网络翻译模型

通过循环网络对源语言文本进行编码，并生成目标语言**翻译结果的过程十分简单**。然而，它仅仅使用一个定长的向量  $\mathbf{h}_m$  编码整个源语言序列。这对于较短的源语言文本没有什么问题，但随着文本序列长度的逐渐加长，**单一的一个向量  $\mathbf{h}_m$  可能不足以承载源语言序列当中的所有信息**。



蓝色的线代表上述简单循环神经网络性能随源语言文本长度的变化趋势。**当文本长度在20个单词以内时**，单一向量能够承载源语言文本中的必要信息。**随着文本序列的进一步增加**，翻译性能的评价指标BLEU的值就开始出现明显地下降。

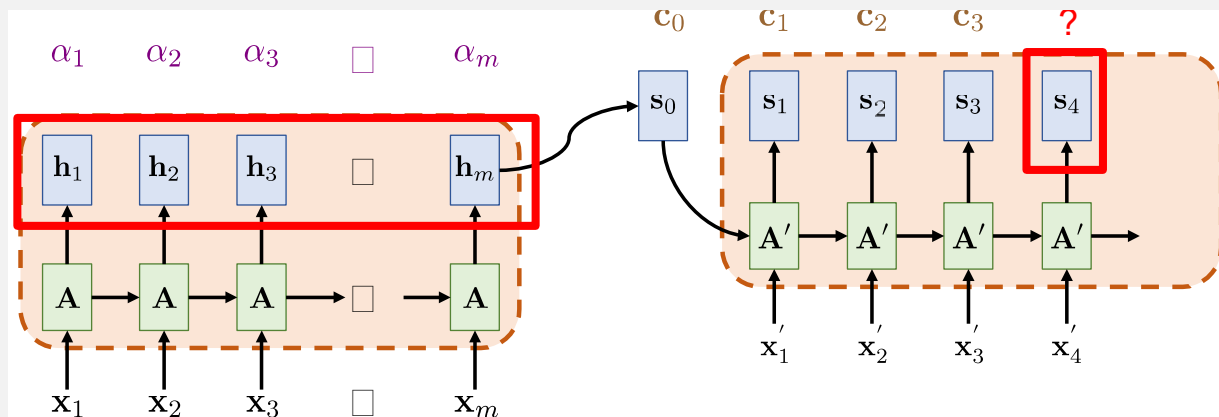






# 循环神经网络翻译模型

引入注意力机制的循环机器翻译架构与基于简单循环网络的机器翻译模型大体结构相似，均采用循环神经网络作为编码器与解码器的实现。**关键的不同点**在于注意力机制的引入使得**不再需要把原始文本中的所有必要信息压缩到一个向量当中**。引入注意力机制的循环机器翻译架构如下图所示。





# 循环神经网络翻译模型

具体来说，给定源语言序列经过编码器输出的向量序列  $h_1, h_2, \dots, h_m$ ，注意力机制旨在依据解码端翻译的需要，自适应地从这个向量序列中查找对应的信息。

与简单循环网络相类似，在  $t$  时刻，旨在通过  $t-1$  时刻的隐状态  $s_{t-1}$  以及  $t$  时刻的输入  $x'_t$  维护循环隐状态并生成当前时刻目标语言翻译结果  $x'_{t+1}$ 。

为了更高效地考虑源语言上下文语义来提高翻译质量，注意力机制通过通过计算一组匹配分数  $\{\text{score}_i\}_{i=1}^m$  并利用softmax归一化为一组权重  $\{\alpha_i\}_{i=1}^m$  自适应地确定源语言中需要聚焦的部分。具体计算公式如下：

$$\{\alpha_i\}_{i=1}^m = \text{Softmax}(\{\text{score}_i\}_{i=1}^m)$$
$$\text{score}_i = \begin{cases} s_{t-1} h_i^T & \text{向量乘} \\ \cos(s_{t-1}, h_i^T) & \text{向量夹角} \\ s_{t-1} W h_i^T & \text{线性模型} \\ \tanh(W[s_{t-1}, h_i]) v^T & \text{单层网络} \end{cases}$$





# 循环神经网络翻译模型

基于上述权重能够得到生成译文  $x'_t$  所必要的源语言信息  $c_t$ ，进一步地，可以将这部分源语言信息与当前时刻的输入  $x'_t$  拼接送入RNN作为新的输入：

$$c_t = \sum_i \alpha_i h_i$$

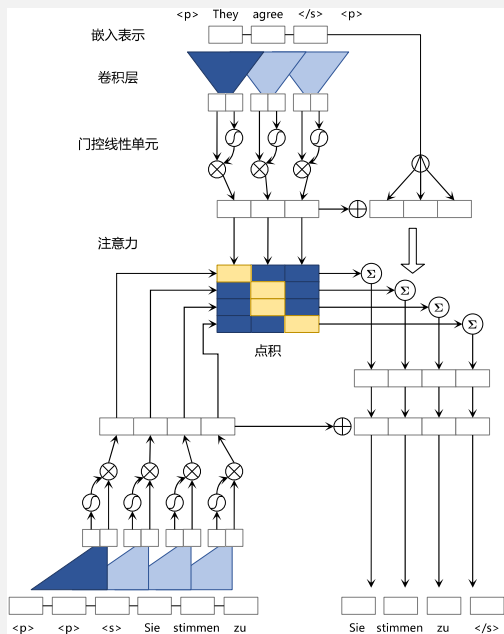
$$s_t = \tanh([x'_t, c_t]U + s_{t-1}W)$$

通过这样的修改，在维护RNN任意时刻隐藏状态并生成译文的过程中，能够自适应地考虑源语言中的哪部分信息需要被聚焦，从而生成更加高质量的译文。



# 卷积神经网络翻译模型

卷积神经网络也是一种经典的神经网络结构，被广泛地使用在自然语言处理的各项任务当中。相较于循环神经网络来说，卷积神经网络**每一步卷积操作并不依赖于前一时间步的计算结果**，因而能够充分并行化以更好地利用GPU的计算资源。



ConvS2S的整体结构如左图所示，它采用卷积神经网络作为**编码器**（图片上侧）和**解码器**（图片左下侧）的具体实现，完成对源语言和目标语言的特征提取。

这种模型结构使得**每一层的网络计算可以完全并行化**。

通过堆叠多层卷积结构，上下文窗口的范围得以不断扩大，从而逐渐建模输入文本中的远距离依赖。



# 卷积神经网络翻译模型

ConvS2S主要由下述几个部件构成：

- **位置编码：**由于ConvS2S摒弃了循环结构，因此需要在输入层引入位置编码来标识输入序列中词与词之间的相对位置关系。
- **卷积层与门控线性单元：**这部分是编码器与解码器的实现模块，分别用于抽取源语言和目标语言的上下文语义特征。
- **残差连接：**编码器和解码器中堆叠的多层卷积结构当中增加了残差连接结构，直接连接每一层的输入与输出，从而提高信息传播效率，减小模型的优化难度。
- **多步注意力机制：**与循环神经机器翻译模型类似，也采用注意力机制自适应地从源语言端检索译文对应的源语言信息。不同的是，此处的注意力计算在解码器的每一层当中都会出现，因而被称为“多步”注意力。



# 卷积神经网络翻译模型

## 位置编码

由于ConvS2S不再使用基于循环的结构编码输入序列，因此模型失去了对于输入文本中词与词之间相对位置关系的感知。位置编码旨在重新给予模型这部分信息。

为了使得卷积模型能够感知到输入序列中单词的相对位置关系，使用额外的位置嵌入

$P = \{p_1, \dots, p_n\}$  用于标识每个单词在句子中的绝对位置

词嵌入和位置嵌入同时被使用作为编码器和解码器的输入  $E = \{s_1 + p_1, s_2 + p_2, \dots, s_n + p_n\}$



# 卷积神经网络翻译模型

## 门控卷积结构

编码器和解码器均采用门控卷积结构作为建模源语言和目标语言的基本部件，这一部件由序列维度上的一维卷积运算和非线性门控机制结合而成。

**卷积过程**能够有效地建模待处理文本中的**局部上下文信息**，而序列中的**长程依赖问题**则可以通过**多层卷积结构**的堆叠得到缓解。**非线性门控机制**使得我们能够建模输入视野下更加**复杂的依赖关系**。

具体来说，对于嵌入层输入的文本表示  $E \in \mathcal{R}^{n \times f}$ ，通过一个线性映射将维度转换到  $d$  维之后，我们能够得到卷积操作的每个上下文窗口的输入  $X \in \mathcal{R}^{k \times d}$ 。对其进行卷积运算如下：

$$A = X \times W_A + b_A$$

$$B = X \times W_B + b_B$$

对  $X$  进行卷积操作得到对应两组输出  $A, B$  后，基于门控线性单元（Gated Linear Units）的非线性变换被用作激活函数得到最终输出： $h = A \otimes \sigma(B)$



# 卷积神经网络翻译模型

**残差链接 (residual connections)** 被引入到模型构建当中。具体来说，每一层卷积单元的输入被直接连接到输出当中如下所示：

$$h^l = A^l \otimes \sigma(B^l) + h^{l-1}$$





# 卷积神经网络翻译模型

## 多步自注意力机制

解码器同样采用了堆叠的多层门控卷积结构完成对目标语言的解码，并在每一层门控卷积之后**通过注意力机制参考源语言信息**。

以解码器第  $l$  层第  $i$  个时间步的注意力计算为例，为了确定需要参考源语言中的哪部分信息，当前时刻的解码器状态  $z_i^l$  以及前一个时刻解码出的目标语言嵌入  $g_i$  用于作出决策的依据：

$$\begin{aligned}d_i^l &= z_i^l W_d^l + b_d^l + g_i \\z_i^l &= \text{Conv}(s_i^l)\end{aligned}$$

基于当前位置的状态依据  $d_i^l$ ，目标语言位置  $i$  相对源语言第  $j$  个单词的注意力得分可以通过  $a_{ij}^l$  和源语言编码器对应位置的输出  $y_j$  计算得到：

$$a_{ij}^l = \frac{\exp(d_i^l \cdot y_j)}{\sum_{t=1}^n \exp(d_i^l \cdot h_t)}$$

基于上述过程得到的注意力得分，可以对源语言不同位置的信息进行加权整合得到为了预测当前位置目标语言单词所需的依据：

$$c_i^l = \sum_{j=1}^m a_{ij}^l (h_j + e_j)$$



# 卷积神经网络翻译模型

这里的源语言端同时利用了编码器的输出  $h_j$  以及对应位置的输入词嵌入  $e_j$ 。这两者对应着更加全面的源语言信息，在实践中也证明十分有效。基于上述源语言信息，可以得到解码器端第  $l$  层的输出为：

$$s_i^{l+1} = c_i^l + z_i^l$$

上述多步注意力机制中的“**多步**”一词主要从两个方面体现。

- 首先从多层卷积堆叠的角度来说，前一层中通过注意力机制动态地决定哪些相关信息需要被关注并传递到下一层当中，而下一层在计算对源语言不同位置的注意力得分过程中又会考虑到这些信息。
- 从时间步的角度来说，在计算目标语言每个位置  $i$  的注意力分布时，前  $k$  个位置的注意力  $c_{i-k}^{l-1}, \dots, c_i^{l-1}$  历史信息都会作为输入的一部分被考虑。



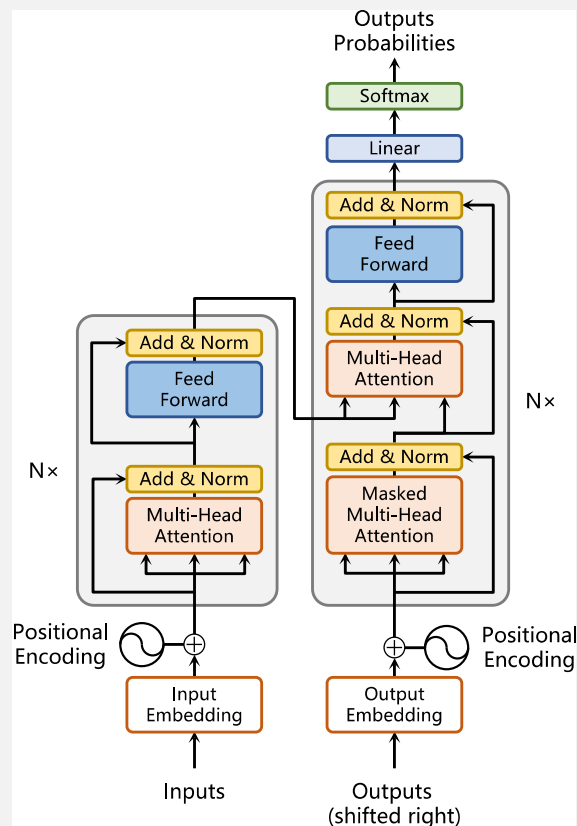
# 自注意力神经网络翻译模型

**Transformer**是由谷歌在2017年提出的一种Seq2Seq架构。它的出现使得机器翻译的性能和效率迈向了一个新的阶段。它摒弃了循环结构，并**完全通过注意力机制完成对源语言序列和目标语言序列全局依赖的建模**。

在抽取每个单词的上下文特征时，Transformer通过自注意力机制（self-attention）衡量上下文中每一个单词对当前单词的重要程度，在这个过程当中没有任何的循环单元参与计算。这种高度可并行化的编码过程使得模型的运行变得十分高效。



# 自注意力神经网络翻译模型

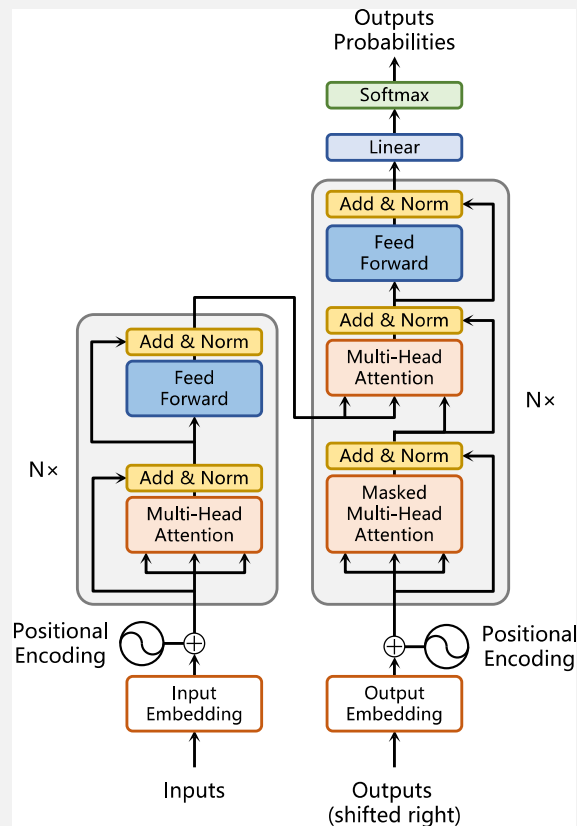


基于Transformer的机器翻译模型架构如左图所示，左侧和右侧分别对应着Seq2Seq模型的编码器和解码器结构。

它们均由若干个基本的Transformer层组成（对应着图中的灰色框）。每个Transformer层都接收一个向量序列  $\{x_i\}_{i=1}^t$  作为输入，并输出一个等长的向量序列作为输出  $\{y_i\}_{i=1}^t$ 。这里的  $x_i$  和  $y_i$  分别对应着文本序列中的一个单词的表示。而  $y_i$  是当前Transformer对输入  $x_i$  进一步整合其上下文语义后对应的输出。在从输入  $\{x_i\}_{i=1}^t$  到输出  $\{y_i\}_{i=1}^t$  的语义抽象过程中，主要涉及如下几个模块：



# 自注意力神经网络翻译模型



**自注意力子层：**对应图中的Multi-Head Attention 部分。使用自注意力机制整合上下文语义，它使得序列中任意两个单词之间的依赖关系可以直接被建模而不基于传统的循环结构，从而更好地解决文本的长程依赖。

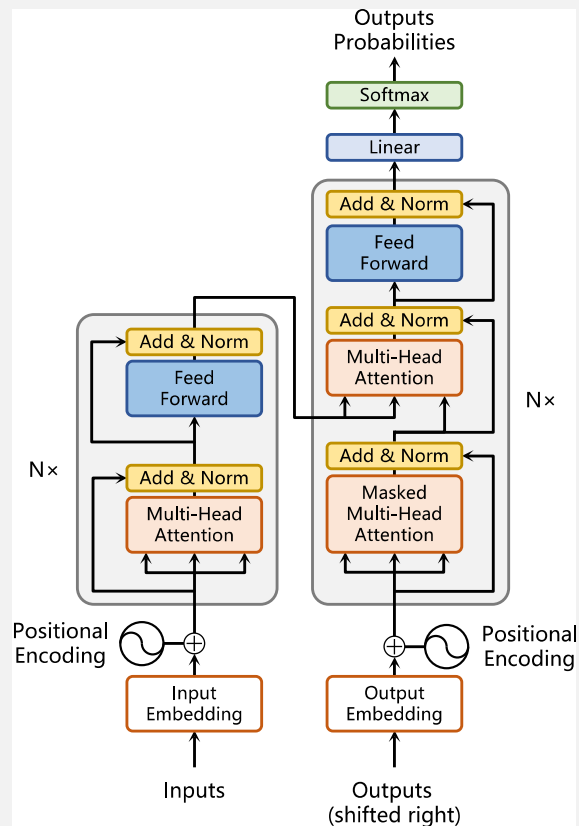
**前馈子层：**对应图中的Feed Forward部分。通过全连接层对输入文本序列中的每个单词表示进行更复杂的变换。

**残差连接：**对应图中的Add部分。它是一条分别作用在上述两个子层当中的直连通路，被用于连接它们的输入与输出。从而使得信息流动更加高效，有利于模型的优化。

**层标准化：**对应图中的Norm部分。作用于上述两个子层的输出表示序列中，对表示序列进行层标准化操作，同样起到稳定优化的作用。



# 自注意力神经网络翻译模型



解码器的每个Transformer层的第一个自注意力子层额外增加了注意力掩码，对应图中的**掩码多头注意力 (Masked Multi-Head Attention)** 部分。

- 额外增加的掩码是用来掩盖后续的文本信息，以防模型在训练阶段直接看到后续的文本序列进而无法得到有效地训练。

解码器端还额外增加了一个**多头注意力 (Multi-Head Attention)** 模块，需要注意的是它同时接收来自编码器端的输出以及当前Transformer层第一个掩码注意力层的输出。

- 它的作用是在翻译的过程当中，为了生成合理的目标语言序列需要观测待翻译的源语言序列是什么。

待翻译的源语言文本，**首先经过编码器端的每个Transformer层**对其上下文语义的层层抽象，最终**输出每一个源语言单词上下文相关的表示**。解码器端以自回归的方式生成目标语言文本，即每个时间步  $t$  参考编码器端输出的所有源语言文本表示以及前  $t-1$  个时刻生成的目标语言文本生成当前时刻的目标语言单词。



# 自注意力神经网络翻译模型

**位置编码：**对于待翻译的文本序列，首先通过输入嵌入层 (Input Embedding) 将每个单词转换为其相对应的向量表示。在送入编码器端建模其上下文语义之前，一个非常重要的操作是在词嵌入中加入位置编码这一特征。

由于Transformer不再使用基于循环的方式建模文本输入，**序列中不再有任何信息能够提示模型单词之间的相对位置关系**。因此补充这部分信息是十分必要的。

具体来说，**序列中每一个单词所在的位置都对应一个实值向量**。这一实值向量会与单词表示对应相加并送入到后续模块中做进一步处理。在训练的过程当中，模型会自动地学习到如何利用这部分位置信息。为了得到不同位置对应的编码，Transformer使用不同频率的正余弦函数如下所示：

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right) \\ \text{PE}(\text{pos}, 2i + 1) &= \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right), \end{aligned}$$

其中，pos表示单词所在的位置， $2i$ 和 $2i+1$ 表示位置编码向量中的对应维度， $d$ 则对应位置编码的总维度。





# 自注意力神经网络翻译模型

**自注意力子层**：自注意力（Self-Attention）机制是基于Transformer的机器翻译模型的基本操作，在源语言的编码和目标语言的生成中频繁地被使用**以建模源语言、目标语言任意两个单词之间的依赖关系**。

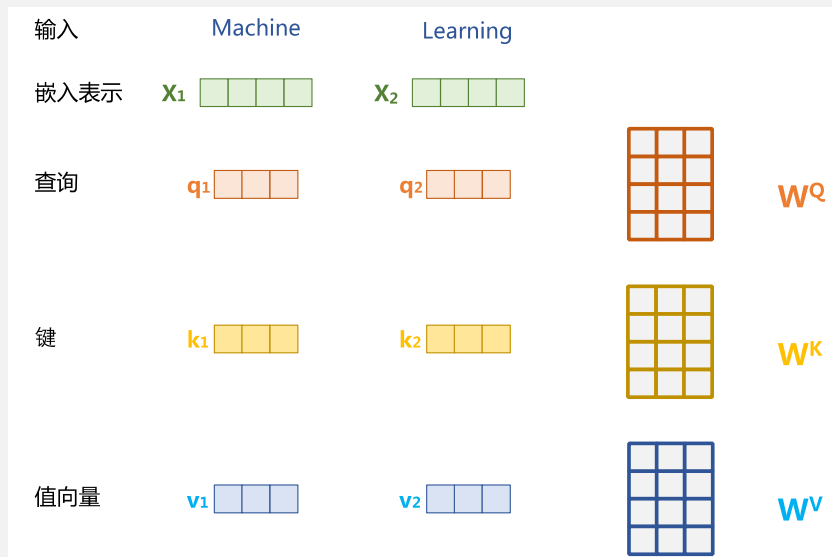
给定由单词语义嵌入及其位置编码叠加得到的输入表示  $\{x_i \in R^d\}_{i=1}^t$ ，为了实现对上下文语义依赖的建模，进一步引入在自注意力机制中**涉及到的三个元素**：查询  $q_i$ （Query），键  $k_i$ （Key），值  $v_i$ （Value）。

在编码输入序列中每一个单词的表示的过程中，这三个元素用于计算上下文单词所对应的权重得分。直观地说，**这些权重反映了在编码当前单词的表示时对于上下文不同部分所需要的关注程度**。





# 自注意力神经网络翻译模型



通过三个线性变换  $W^Q \in \mathbb{R}^{d \times d_k}$ ,  $W^K \in \mathbb{R}^{d \times d_k}$ ,  $W^V \in \mathbb{R}^{d \times d_v}$  将输入序列中的每一个单词表示  $x_i$  转换为其对应向量  $q_i \in \mathbb{R}^{d_k}$ ,  $k_i \in \mathbb{R}^{d_k}$ ,  $v_i \in \mathbb{R}^{d_v}$

为了得到编码单词  $x_i$  时所需要关注的上下文信息，通过位置  $i$  查询向量与其他位置的键向量做点积得到匹配分数： $q_i \cdot k_1, q_i \cdot k_2, \dots, q_i \cdot k_t$

经过Softmax归一化为概率之后，与其他位置的值向量相乘来聚合我们希望关注的上下文信息，并最小化不相关信息的干扰：

$$Z = \text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

$\sqrt{d}$  为放缩因子



# 自注意力神经网络翻译模型

为了进一步增强自注意力机制聚合上下文信息的能力，提出了**多头 (Multi-head) 自注意力的机制**，以从关注上下文的不同侧面。上下文中每一个单词的表示  $x_i$  经过多组线性映射到不同的表示子空间中。

$$\mathbf{Z} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V}$$

利用上述公式，在不同的子空间中分别计算并得到不同的上下文相关的单词序列表示  $\{\mathbf{Z}_j\}_{j=1}^N$

线性变换  $\mathbf{W}^O \in \mathbb{R}^{(Nd_v) \times d}$  用于综合不同子空间中的上下文表示并形成自注意力层最终的输出  $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^t$



# 自注意力神经网络翻译模型

**前馈子层：**前馈子层接受自注意力子层的输出作为输入，并通过一个带有Relu激活函数的两层全连接网络对输入进行更加复杂的非线性变换。实验证明，这一非线性变换会对模型最终的性能产生十分重要的影响。

$$\text{FFN}(\boldsymbol{x}) = \text{Relu}(\boldsymbol{x}\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2$$

$$\text{Relu}(\boldsymbol{x}) = \max(0, \boldsymbol{x}),$$

其中  $\boldsymbol{W}_1, \boldsymbol{b}_1, \boldsymbol{W}_2, \boldsymbol{b}_2$  表示前馈子层的参数。实验结果表明，增大前馈子层隐状态的维度有利于提升最终翻译结果的质量，因此，前馈子层隐状态的维度一般比自注意力子层要大。



# 自注意力神经网络翻译模型

**残差连接与层标准化：**事实上Transformer是一个非常庞大的网络结构。它的编码器和解码器均由6层基本的Transformer层组成，每一层当中都包含复杂的非线性映射，这就导致模型的训练比较困难。因此，研究者在Transformer层中进一步**引入了残差连接与层标准化技术**以进一步**提升训练的稳定性**。具体来说，残差连接主要是指使用一条直连通道直接将对应子层的输入连接到输出上去，从而避免由于网络过深在优化过程中潜在的梯度消失问题：

$$x^{l+1} = f(x^l) + x^l,$$

其中  $x^l$  表示第  $l$  层的输入,  $f(\cdot)$  表示一个映射函数。此外，为了进一步使得每一层的输入输出范围稳定在一个合理的范围内，层标准化技术被进一步引入Transformer的每一层当中：

$$LN(x) = g \cdot \frac{x - \mu}{\sigma} + b$$

其中  $\mu$  和  $\sigma$  分别表示均值和方差，用于将数据平移缩放到均值为 0，方差为 1 的标准分布， $g$  和  $b$  是可学习的参数。层标准化技术可以有效地缓解优化过程中潜在的不稳定、收敛速度慢等问题。



# 大纲

---

- 机器翻译概述
- 基于统计的机器翻译方法
- 基于神经网络的机器翻译方法
- 机器翻译语料库



# 机器翻译语料库

**WMT数据集：**该数据集是一个以英语为主的多语言机器翻译数据集，涉及英中、英德翻译等多种任务，数据来源于新闻、医学、翻译等领域。

**IWSLT数据集：**该数据集是一个来自于TED演讲的文本翻译数据集，语料规模较小，涉及英德、英中翻译等任务。

**NIST数据集：**该数据集是一个新闻翻译领域的高质量数据集，评测集包括4句参考译文，涉及中英、英捷翻译等任务。

**TVsub：**该数据集是一个抽取自电视剧字幕的翻译数据集，适合于对话中的长距离上下文依赖研究，涉及中英翻译任务。

**Flickr30k：**该数据集是多模态机器翻译的主流数据集之一，包含31783张图片，每张图片对应5个语句标注，涉及英德翻译任务。

**Multi30k：**该数据集是多模态机器翻译的主流数据集之一，包含31014张图片，每张图片对应5个语句标注，设计英德、英法翻译任务。

**IAPRTC-12：**该数据集是多模态翻译的主流数据集之一，包含20000张图片及其对应标注，涉及英德翻译任务。

**IKEA：**该数据集是多模态翻译领域的主流数据集之一，包含3600张图片及其对应标注，涉及英德、英法翻译任务。



# 小结

---

- 机器翻译概述
- 基于统计的机器翻译方法
- 基于神经网络的机器翻译方法
- 机器翻译语料库